Introducción

La inteligencia artificial (IA) es hoy una de las tecnologías más influyentes en el desarrollo de sistemas automatizados. Su capacidad para **procesar información, aprender patrones y tomar decisiones** permite transformar procesos que antes dependían exclusivamente de reglas fijas y programación manual.

Objetivos de la lecture 🚀

1. Explicar las **diferencias entre IA y la automatización** basada en reglas, y cómo la IA mejora la adaptabilidad de los sistemas automatizados.
2. Reconocer y definir los **principales componentes de un modelo de lenguaje extenso (LLM)** y cómo interactuar con ellos.
3. Comprender el impacto de los **parámetros técnicos, como tokens, temperature y max_tokens**, al interactuar con los modelos de IA.
4. Diseñar **prompts efectivos y estructurados** para obtener respuestas claras y útiles, aplicando técnicas avanzadas como Chain of Thought y Few-shot Learning.
5. Comparar **plataformas de IA** y elegir el modelo adecuado según el caso de uso y los costos asociados.

Conceptos Esenciales de Inteligencia Artificial y Tipos de Aplicación

La **Inteligencia Artificial (IA)** es un concepto cada vez más presente en nuestra vida cotidiana, tanto en el trabajo y la educación, hasta el entretenimiento, la atención médica y actividades cotidianas. Comprender **qué es la IA y cómo se diferencia de la automatización tradicional** resulta clave para integrarla de forma efectiva en **workflows, procesos y soluciones empresariales.**

¿Qué es la Inteligencia Artificial?

La Inteligencia Artificial *se puede definir como* ***la capacidad de un sistema informático para procesar datos, aprender patrones a partir de ellos y generar respuestas o acciones inteligentes****, imitando capacidades humanas como razonar, conversar, predecir o clasificar información.*

A diferencia de los sistemas convencionales de automatización (como la Automatización de Procesos Robóticos, RPA), que funcionan con reglas fijas ("si pasa A, hacer B"), la **IA aprende de los datos y puede adaptarse a contextos nuevos**. Esto

le permite resolver problemas no previstos ante situaciones nuevas, basándose en lo aprendido de grandes volúmenes de datos.

Ejemplos cotidianos de IA en acción

La IA ya es parte de una gran cantidad de servicios y productos que usamos día a día, aunque no siempre lo notemos. Algunos ejemplos incluyen:

- **Recomendaciones en Spotify, Netflix o YouTube:** Los algoritmos analizan tu historial de actividad y sugieren música, películas o videos basándose en tus gustos previos, comparandolos con patrones de otros usuarios similares para sugerir contenido relevante.
- **Filtros de spam y respuestas inteligentes en Gmail:** Los modelos de IA detectan patrones en correos sospechosos y filtran los mensajes no deseados, además de proponer respuestas rápidas para mensajes frecuentes.
- **Chatbots de atención al cliente:** Capaces de comprender consultas comunes y dar respuestas automáticas, liberando tiempo de agentes humanos.
- **Predicciones de texto en dispositivos móviles:** Al escribir un mensaje, la IA predice la siguiente palabra o frase, lo que acelera la comunicación.

Estos ejemplos muestran cómo la IA procesa grandes volumenes de datos para ofrecer soluciones personalizadas y eficientes.

IA vs Automatización Tradicional

Antes de la llegada de la IA moderna, la automatización en los procesos empresariales se basaba en reglas predefinidas . Estos sistemas ejecutaban acciones sólo cuando se cumplían condiciones exactas, sin capacidad de aprendizaje o adaptación.

Característica	Automatización Tradicional (Reglas Fijas)	Inteligencia Artificial (Modelos Inteligentes)
Tipo de lógica	Basada en instrucciones del tipo "si ocurre X, hacer Y".	Aprende de los datos y reconoce patrones complejos.
Adaptabilidad	No aprende ni se ajusta a nuevos escenarios.	Se adapta y mejora con la experiencia.
Tipo de entrada	Datos estructurados y repetitivos.	Datos diversos, complejos o no estructurados.
Ejemplo	Enviar un correo cuando llega una factura con monto mayor a $1,000.	Detectar facturas sospechosas por fraude analizando múltiples patrones.

Las automatizaciones tradicionales funcionan mejor en **procesos predecibles y repetitivos, con reglas claras y condiciones estables**, como la generación automática de reportes o la actualización de inventarios.

En cambio, los modelos de IA son más adecuados cuando **el entorno es variable y los datos son complejos o ambiguos**. Son ideales para tareas que requieren análisis y adaptación, como predecir el comportamiento de clientes o detectar fallos en sistemas industriales.

Tipos de IA aplicada según su función

Dentro de la IA existen **varios enfoques** según el tipo de tarea que resuelven. Los más frecuentes en entornos de automatización son:

- **Clasificación:** Permite asignar categorías o etiquetas a los datos de entrada.

Ejemplo: Determinar si un mensaje de cliente es una queja, una consulta o un agradecimiento.

- **Predicción:** Estima valores o comportamientos futuros basándose en información histórica.

Ejemplo: Prever si un cliente hará una compra, estimar la demanda de un producto o cuándo ocurrirá un incidente.

- **Generación:** Crea contenido original a partir de datos existentes o descripciones.

Ejemplo: Redactar automáticamente respuestas a emails, escribir resúmenes o generar imágenes/texto a partir de descripciones.

- **Análisis (Descriptiva):** Descubre patrones, tendencias o relaciones dentro de grandes volúmenes de datos.

Ejemplo: Identificar tendencias en opiniones de usuarios, analizar sentimientos en comentarios de redes sociales o identificar temas frecuentes en reseñas de productos.

- **Optimización:** Mejora procesos, recursos o decisiones buscando la solución más eficiente.

Ejemplo: Organizar el recorrido de los repartidores para ahorrar tiempo o ajustar la cantidad de materiales que se usan en producción.

- **Procesamiento del Lenguaje Natural (NLP):** Comprende, interpreta y genera lenguaje humano.

Ejemplo: Chatbots de atención al cliente, traducción automática de textos, asistentes virtuales como Alexa o Google Assistant.

- **Visión Artificial (Computer Vision):** Interpreta y analiza imágenes o videos para extraer información útil.

Ejemplo:**** Contar cajas en un almacén automáticamente mediante cámaras, detección de campos y sellos en documentos.

- **IA Conversacional:** Interactúa con usuarios a través de texto o voz de forma natural.

Ejemplo: Asistentes virtuales que responden preguntas, sistemas de soporte al cliente automatizado, aplicaciones educativas interactivas.

- **Recomendación:** Sugiere opciones o decisiones personalizadas según preferencias o patrones de comportamiento.

Ejemplo: Recomendaciones de películas en Netflix, productos en Amazon, música en Spotify o contenido personalizado en redes sociales.
A medida que los volúmenes de datos crecen y los entornos se vuelven más dinámicos, los sistemas puramente basados en reglas resultan insuficientes. Las organizaciones necesitan herramientas que **aprendan, se adapten y mejoren de forma continua**, y es allí donde la IA ofrece una ventaja competitiva clave.

En la siguiente sección profundizaremos en los **Modelos de Lenguaje Extenso (LLMs)**, que son el "cerebro" de muchas soluciones modernas de IA generativa, y entenderemos sus componentes básicos, capacidades y limitaciones. Esto te dará las herramientas fundamentales para aprovechar estas tecnologías en tus propios workflows y procesos automatizados.

Modelos de Lenguaje Extenso (LLM): Arquitectura y Capacidades

Los **Modelos de Lenguaje Extenso (LLM, por sus siglas en inglés Large Language Models)** son una de las tecnologías más influyentes dentro de la inteligencia artificial generativa moderna. Su capacidad para comprender, procesar y generar lenguaje natural con fluidez los convierte en una herramienta fundamental en entornos de automatización, productividad y análisis inteligente de datos.

En esta sección, comprenderás qué es un LLM, cuáles son sus componentes básicos, por qué son relevantes para la automatización y qué capacidades y límites presentan hoy en día.

¿Qué es un LLM y por qué son importantes para la automatización e IA generativa?

Un LLM ***es un modelo de inteligencia artificial entrenado con enormes volúmenes de datos textuales*** *(libros, artículos, páginas web, código, documentación técnica, entre otros) para comprender, procesar y generar lenguaje natural (texto) de forma coherente y contextual*.

Su función principal es **predecir la siguiente palabra en una secuencia**, lo que le permite estructurar ideas, responder preguntas y generar contenido en lenguaje natural.

Su importancia dentro de la automatización e IA generativa radica en que los LLMs permiten que los sistemas:

- **Comprendan** instrucciones en lenguaje natural y las transformen en acciones automatizadas.
- **Extraigan** información clave de documentos o mensajes y tomen decisiones basadas en ella.
- **Generen** textos o resúmenes adaptados al contexto del usuario o del flujo de trabajo.
- **Integren** adaptabilidad y aprendizaje continuo en procesos que antes eran puramente rígidos o basados en reglas.

¿Cómo funcionan los LLM?

Durante su entrenamiento, **los LLM aprenden patrones lingüísticos a partir de millones de ejemplos**: cómo se estructuran las oraciones, qué palabras suelen aparecer juntas y las relaciones semánticas entre conceptos.

Aunque **no tienen conciencia ni comprensión real**, simulan entendimiento al predecir la siguiente palabra más probable en una secuencia, generando resultados que parecen inteligentes y naturales.

El proceso de funcionamiento de un LLM se basa en **tres componentes** clave:

1. **Embeddings:** Representan palabras o frases como **vectores** que reflejan significado y relaciones semánticas (por ejemplo, “automóvil” y “vehículo” son vectores cercanos).
2. **Mecanismo de Atención:** Permite que el modelo **enfoque su “atención”** en las partes más relevantes del texto al generar una respuesta, priorizando la información útil.
3. **Arquitectura Transformer:** Combina **capas de atención y aprendizaje profundo** para procesar textos largos y mantener coherencia contextual.

Una vez entrenado, **el modelo puede generar respuestas coherentes ante nuevas entradas, basándose en su experiencia previa**. El modelo aprende de patrones lingüísticos y puede generar texto con coherencia contextual, tono adecuado y consistencia estructural.

Capacidades y resultados esperables de los LLM

Los LLM pueden realizar numerosas tareas relacionadas con el procesamiento del lenguaje natural, como:

- **Generar** texto coherente y relevante en diferentes estilos (por ejemplo, correos automáticos, respuestas a clientes).
- **Resumir, clasificar y analizar** grandes volúmenes de texto en segundos.
- **Traducir y reescribir** información adaptándose al tono, idioma y contexto del usuario.
- **Responder** preguntas sobre documentos extensos o combinar información dispersa de múltiples fuentes.

Estas capacidades los convierten en herramientas ideales para automatizar procesos que requieren comprensión textual o generación de lenguaje natural.

Para pensar 🔍

¿Qué tipo de tareas, en tus flujos de trabajo diarios, se benefician más si una "inteligencia" pudiera comprender textos y generar instrucciones personalizadas?

Por ejemplo: atender correos repetitivos, clasificar reportes, resumir conversaciones de clientes.

Limitaciones típicas de los LLM

Aunque los LLM han demostrado capacidades notables, **presentan limitaciones inherentes** que deben considerarse para diseñar sistemas confiables.

A continuación, se detallan las limitaciones más relevantes y cómo pueden afectar el desempeño de una automatización:

Limitación	Efecto	Impacto en la práctica	Mitigación recomendada
Memoria limitada (context window)	Solo pueden procesar una cantidad finita de texto a la vez.	Puede omitir detalles importantes en textos extensos o perder coherencia en largas conversaciones.	Dividir el contenido en fragmentos, usar resúmenes intermedios o emplear modelos con contexto ampliado.
Alucinaciones	El modelo genera información incorrecta o inventada al no tener contexto o datos actualizados.	Riesgo de decisiones erróneas en entornos sensibles si no se valida la información.	Validar resultados con fuentes externas o aplicar reglas de verificación posterior (human-in-the-loop).
Dependencia del prompt	La calidad de la salida depende directamente de cómo se formula el prompt.	Prompts ambiguos o imprecisos generan respuestas incompletas o fuera de tema.	Diseñar prompts claros, estructurados y con ejemplos de formato esperado.
Sesgos y desactualización	Los modelos reflejan los sesgos y limitaciones de los datos con los que fueron entrenados	Pueden reproducir desigualdades o información obsoleta.	Añadir bases de conocimiento al Modelo, aplicar filtros de contenido y revisar resultados.
Capacidades matemáticas limitadas	Los LLM no realizan cálculos con precisión simbólica; sólo "simulan" operaciones mediante patrones aprendidos.	Responden bien a cálculos simples, pero pueden fallar en operaciones complejas o razonamiento lógico matemático.	Para operaciones precisas, combinar la IA con herramientas externas (por ejemplo, Python, Excel, WolframAlpha)
Falta de comprensión real del contexto	Los modelos no entienden el significado del texto, sólo correlaciones entre palabras	Pueden dar respuestas coherentes, pero sin fundamento lógico.	Implementar supervisión humana y procesos de human-in-the-loop en decisiones críticas.

Estas **limitaciones pueden afectar la confiabilidad, precisión, eficiencia y transparencia** del sistema. Pueden surgir **alucinaciones, desactualización, necesidad de instrucciones excesivas o falta de trazabilidad** en las respuestas.

Aunque no invalidan su uso, r**equieren mitigación** mediante un contexto bien definido, prompts claros, conexión con datos actualizados y supervisión humana en tareas críticas para mantener la confiabilidad y eficiencia del sistema.

Evolución de los LLM: Principales generaciones y modelos

Con el avance de la IA se han ido desarrollando diferentes familias de LLM, cada una aportando capacidades nuevas o distintas especializaciones.

Entre los **LLMs** más destacados encontramos:

Modelo	Desarrollador	Fortaleza principal	Casos de uso comunes
GPT-3	OpenAI	Generación de texto coherente a gran escala.	Redacción, chatbots, creación de contenido.
GPT-4	OpenAI	Razonamiento avanzado y comprensión de contexto extendido.	Atención al cliente, análisis de datos y generación de reportes.
GPT-5	OpenAI	Integración multimodal (texto, imagen, audio), razonamiento estructurado y memoria dinámica.	Asistentes conversacionales complejos, análisis de contexto empresarial, creación de contenido.
Claude 3	Anthropic	Procesamiento de contextos muy extensos y enfoque en seguridad.	Redacción técnica, revisión de código, tareas de compliance.
Gemini 1.5	Google	Integración con Workspace, razonamiento en múltiples idiomas.	Tareas administrativas, resúmenes corporativos y soporte interno.
Gemini 2.5	Google DeepMind	Integración nativa con el ecosistema Google y análisis multimodal (texto, imagen, video).	Automatización empresarial, análisis de documentos, educación y búsqueda inteligente.
Llama 3	Meta	Código abierto y adaptable para investigación.	Implementaciones personalizadas, investigación y despliegue local.
DeepSeek R1	DeepSeek	Balance entre rendimiento y costo.	Alternativa empresarial a modelos comerciales.
Modelos especializados (Whisper, DALL-E, Code Llama)	Varios	Enfoque en tareas específicas (voz, imagen o código).	Transcripción, generación de arte digital o programación asistida.

Los Modelos de Lenguaje Extenso **han revolucionado la forma en que interactuamos con la información**. Sin embargo, su implementación exige comprender tanto sus capacidades como sus limitaciones.

En las próximas secciones exploraremos **cómo integrar estos modelos dentro de plataformas de automatización y flujos de trabajo**, evaluando cuál conviene según el tipo de tarea, contexto y recursos disponibles.

Plataformas y APIs de IA: Acceso, Comparación y Parámetros Clave

Comprender cómo seleccionar y usar plataformas de IA, así como interpretar y ajustar los parámetros fundamentales de los modelos, es esencial para cualquier proceso de automatización moderna basada en IA.

En esta sección aprenderás **cómo funcionan las APIs que dan acceso a modelos avanzados, cuáles son las diferencias entre plataformas, y qué parámetros son los más importantes para controlar el comportamiento de un modelo**.

Acceso a Modelos de IA: Plataformas y APIs

En la mayoría de los casos, los modelos de IA se integran en aplicaciones de automatización a través de APIs (interfaces de programación de aplicaciones). Estas APIs actúan como un puente entre tu aplicación y el modelo, enviando instrucciones (prompts) y recibiendo respuestas procesadas.

Existen diferentes tipos de plataformas que ofrecen acceso a LLMs y modelos especializados:

- **Plataformas Unificadas:**

OpenRouter: Es una plataforma que centraliza el acceso a múltiples modelos (OpenAI, Anthropic, Google, modelos open source) desde una sola API.

Esto facilita la comparación de modelos, la gestión centralizada de Keys y gastos, y reduce la complejidad del desarrollo. Es cuando se requiere experimentación con distintos modelos.

- **APIs Directas de Proveedores:**

Permiten conectarse directamente con el servicio de un proveedor específico:

- **OpenAI**: Acceso directo a modelos como GPT-3.5 y GPT-4.
- **Anthropic**: APIs para acceder a modelos como Claude.
- **Google AI Studio**: Ofrece acceso a Gemini y otros modelos de Google.

Estas APIs son ideales para integraciones directas, donde se requiere estabilidad y soporte oficial del proveedor.

- **Modelos Open Source y Especializados:**

Algunas alternativas permiten autoalojar los modelos en servidores propios, lo que ofrece mayor control y reducción de costos a largo plazo.

- **Llama**: Modelo de Meta de código abierto, ideal para “self-hosting” (ejecución local) y mayor control.
- **DeepSeek**: Alternativa eficiente y económica.
- **Whisper, DALL-E**: Modelos especializados para audio (Whisper) e imágenes (DALL-E).

*Estas opciones permiten **personalizar e incluso autoalojar los modelos**, aunque requieren mayor infraestructura técnica.*

Parámetros Esenciales para Controlar la Salida

Cuando interactuamos con un modelo mediante una API o un playground, podemos ajustar **ciertos parámetros que modifican cómo interpreta el prompt y qué tan variada o controlada será la respuesta.**

Tokens

Los tokens son fragmentos de texto (palabras o partes de palabras). En promedio 1 token ≈ 4 caracteres en inglés, o 2-3 en español. El costo de uso depende del número total de tokens (entrada + salida). Afecta directamente el costo y el límite de texto procesado.

Se puede estimar la cantidad de tokens utilizando calculadoras online. Algunas opciones para probar son:

- Token counter: https://tokencounter.org/es(opens in a new tab)
- Toolslab: https://toolslab.dev/es/tools/ai-prompt-token-counter(opens in a new tab)
- calculadora now: https://calculadora.now/calculadora-de-tokens-de-ia/(opens in a new tab)
- Calculadora de precios de Google Cloud: https://cloud.google.com/products/calculator

Context Window (Ventana de Contexto)

Es la memoria temporal del modelo: cuántos tokens puede tener presentes. Si superamos ese límite, el modelo “olvida” partes anteriores del diálogo.

Temperature

Controla el nivel de creatividad o aleatoriedad en las respuestas:

- 0,0–0,3: resultados más predecibles, ideal para tareas analíticas o de datos.

- 0,7–1,0: resultados más creativos, útil para redacción, ideación o diseño de prompts.

Ejemplo de Prompt: “Describe un automóvil.”

- Temperature 0.2 = “Un automóvil es un vehículo motorizado de cuatro ruedas usado para transporte.”
- Temperature 0.9 = “Un automóvil es como una cápsula de libertad sobre ruedas, lista para devorar kilómetros.”

Max Tokens

Define el número máximo de tokens en la respuesta del modelo. Es útil para controlar la extensión de la salida y evitar costos excesivos.

System Prompt y User Prompt

Uno de los elementos más importantes es el **prompt**, que representa la instrucción que guía al modelo. Sin embargo, existen dos tipos fundamentales que cumplen funciones distintas (ver cuadro).

El modelo combina ambas instrucciones para producir una respuesta coherente.
El ***System Prompt*** actúa como el “manual interno” del asistente. Mientras el ***User Prompt*** representa la “orden directa” que recibe en cada interacción.

Tipo de Prompt	Función	Ejemplo	Resultado
System Prompt	Define el *rol* y el *comportamiento base* del modelo.	"Eres un asistente técnico especializado en automatización con Python. Responde de forma concisa y profesional."	Todas las respuestas se ajustarán a ese perfil, sin importar la pregunta.
User Prompt	Es la instrucción o consulta puntual del usuario.	"Escribe un script que automatice el envío de correos con reportes de ventas."	El modelo responderá siguiendo las reglas del *system prompt*.

*La **configuración adecuada de estos parámetros** permite controlar el comportamiento del modelo según las necesidades del workflow automatizado.*

¿Qué Factores Importan para seleccionar un modelo de IA?

Antes de elegir una API o modelo, es necesario comparar ciertos aspectos clave que afectan directamente el rendimiento. Entre ellos, encontramos los siguientes:

Tipo de Tarea

Es fundamental seleccionar el modelo adecuado según el tipo de tarea que se va a realizar, ya que **algunos modelos están específicamente optimizados para ciertos usos**. Por ejemplo, GPT-4 o Claude 3 pueden realizar desde redacción hasta razonamiento complejo, mientras que Whisper se especializa únicamente en transcribir audio.

Por otro lado, muchos modelos modernos incluyen **capacidades multimodales, lo que significa que pueden procesar no solo texto, sino también imágenes, audio o incluso video**. Esto es particularmente útil para tareas complejas que requieren información de diferentes tipos de medios simultáneamente.

Algunos **ejemplos** de modelos multimodales incluyen CLIP (de OpenAI), que combina texto e imágenes, o Flamingo (de DeepMind), que integra texto, imágenes y video.

Ventana de Contexto ("Context Window")

Indica **cuántos tokens (información) puede considerar el modelo en una sola petición**. A medida que **aumenta el número de tokens en un prompt, la calidad de la respuesta puede verse afectada, ya que el modelo tiene que manejar más información**. Para tareas que requieren un análisis profundo o la comprensión de documentos largos (como resúmenes extensos o análisis detallados), es recomendable optar por modelos con una ventana de contexto amplia.

Por ejemplo, GPT-3.5 tiene una ventana de contexto de 4K tokens, mientras que GPT-4 puede manejar entre 8K y 128K tokens, dependiendo de la versión. De manera similar, los modelos Gemini 2.5 también permiten un número mayor de tokens, lo que los hace adecuados para contextos más largos.

Costo

El **coste de utilizar un modelo de IA varía según el tipo de modelo**, la cantidad de **tokens** procesados (entrada + salida), y la **plataforma** elegida.

Por ejemplo, OpenRouter permite comparar los precios entre diferentes modelos para facilitar la toma de decisiones. Por otro lado, Google ofrece herramientas para calcular tanto el coste del uso de la IA como la infraestructura necesaria para ejecutarlos, lo que facilita una planificación más precisa de los gastos. Por otro lado, algunas plataformas open source pueden ejecutarse localmente, lo que elimina los costes asociados a las API, pero requieren una infraestructura propia. Por ejemplo, LLaMA de Meta.

Integración y Flexibilidad

Las **APIs,** como OpenRouter, **simplifican la integración** al permitir cambiar de modelo sin necesidad de reescribir el código base. Es útil cuando se requiere experimentar con diferentes modelos sin comprometer la infraestructura existente.

Además, algunos proveedores **ofrecen herramientas adicionales para facilitar la experimentación** como Google o Anthropic, con interfaces visuales (playgrounds) que permiten probar el modelo de manera interactiva y ajustar parámetros de forma más intuitiva.

Herramientas y Playground para Experimenta

Casi todas las plataformas ofrecen interfaces visuales **("playgrounds")** para experimentar con prompts y observar de inmediato cómo distintos parámetros cambian la salida. OpenAI, Anthropic y Google AI Studio proveen sus propios playgrounds; **OpenRouter agrega la ventaja de probar varios modelos desde un solo lugar.**

En la práctica, los pasos básicos para experimentar suelen ser:

Paso 1
Elegir modelo y proveedor en el playground.

Paso 2
Escribir el prompt.

Paso 3
Configurar parámetros clave: temperature, max_tokens, system prompt.

Paso 4
Ejecutar y observar el resultado.

Paso 5
Comparar resultados al cambiar uno o varios parámetros.

Playgrounds propuestos:

- Playground de Google AI Studio (https://aistudio.google.com/welcome(opens in a new tab))
- Playground de OpenRouter (https://openrouter.ai/(opens in a new tab))

Mini-práctica: Variación de Temperature y Análisis de Resultados

Práctica con AI Playgrounds

En esta mini práctica utilizarás alguno de los playgrounds propuestos (Google AI Studio o OpenRouter)

1. **Ingresa el siguiente user prompt:** "Resume en una frase: El océano es fundamental para la vida en la Tierra y regula el clima global."
2. **Ejecuta el prompt dos veces:**
    - Primer intento con temperature = **0.0**
    - Segundo intento con temperature = **0.9**
3. **Observa las respuestas y responde:** ¿Qué diferencias notas en la creatividad o la precisión de las salidas?
4. **Posibles soluciones:**
    - **Con temperature = 0.0**, la respuesta será muy similar en cada ejecución y prioriza la literalidad del texto original (ejemplo: “El océano regula el clima global y es vital para la vida en la Tierra.”).
    - **Con temperature = 0.9**, el modelo puede ofrecer variaciones más creativas y menos convencionales (ejemplo: “El océano, motor del clima y fuente de vida, sostiene a nuestro planeta.”).

Demo práctica: Explorando parámetros y comparando modelos

En este video vas a descrubrir en tiempo real cómo cambian las respuestas de un modelo al modificar tokens, temperature y plataforma, mientras aprendés a documentar tus primeras pruebas como un profesional.

Prompt Engineering

Hasta ahora, has aprendido cómo funcionan los modelos de IA y cómo seleccionarlos según cada caso de uso. Ahora, aprenderás a **diseñar prompts efectivos** para que los modelos de lenguaje generen respuestas claras, precisas y útiles. Además, vas a dominar técnicas avanzadas de prompting que te permitirán controlar el razonamiento, la creatividad y el formato de salida de la IA.

Cuando interactúas con un modelo de lenguaje, el prompt,es decir, lo que escribís para pedirle algo, es tan importante como la propia capacidad del modelo. Podés pensar en el prompt como la **interfaz natural entre tu pensamiento y la inteligencia artificial**. Si sabés estructurarlo correctamente, podés transformar una idea vaga en una respuesta precisa, coherente y accionable.

Aprender a escribir buenos prompts no es solo una habilidad técnica, sino también una competencia comunicacional ya que estás aprendiendo a conversar con una mente sintética.

Para ello, vamos a ver primero cómo se compone un buen prompt, después avanzaremos hacia técnicas más sofisticadas que te permitirán modelar el comportamiento del sistema, y finalmente entenderemos cómo estructurar las salidas para que puedan integrarse en procesos automatizados.

Pero antes veamos **2 tipos de prompt:**

- **Simple**
    - **<u>Ejemplo:</u>**

"Haz un resumen del informe anual de sostenibilidad de nuestra empresa XYZ."

- **<u>Resultado esperado:</u>**

Un párrafo breve que resume el informe, pero sin estructura clara ni enfoque estratégico.

- **Estructurado con Prompt Engineering**
    - **<u>Ejemplo:</u>**
        - **Rol**: Analista senior en sostenibilidad corporativa.
        - **Contexto**: Informe anual de sostenibilidad de la empresa XYZ, incluye datos de impacto ambiental, social y de gobernanza (ESG).
        - **Tarea**: Identifica los 5 hallazgos más importantes del informe y sus implicaciones estratégicas.
        - **Formato**: Tabla con columnas: Hallazgo, Impacto, Recomendación.

        - **Estilo**: Profesional, conciso y adecuado para presentación ante la junta directiva.

    - **Resultado esperado:**

Una tabla organizada que resuma los hallazgos clave del informe, mostrando impacto y recomendaciones estratégicas de forma clara y profesional, lista para ser presentada a directivos o inversionistas.

Estructura de un Prompt Efectivo

Un prompt efectivo no surge por casualidad. Es el resultado de una combinación precisa de varios elementos que orientan al modelo de lenguaje para entregar respuestas claras, útiles y coherentes. Así como una receta bien definida garantiza un platillo consistente, **un prompt estructurado de manera adecuada produce resultados predecibles, que pueden ser integrados eficientemente en procesos productivos.**

¿Por qué estructurar un prompt?

Cuando usamos IA para tareas en producción, no solo para experimentar, **la precisión y la reproducibilidad son fundamentales**. Los modelos necesitan instrucciones claras sobre **qué rol asumir, qué información considerar, qué tarea realizar y, clave para automatización, en qué formato entregar la respuesta**. Si uno de estos componentes falta o es ambiguo, el output puede volverse errático o poco útil.

Por ejemplo, si le pides a alguien que "haga una presentación", sin especificar detalles como el tema, el formato, la extensión o la audiencia, es muy probable que el resultado no se ajuste a lo que necesitas. Lo mismo sucede con los prompts y sus resultados.

Componentes principales de un prompt efectivo

A continuación, se explican los elementos básicos que conforman un prompt robusto. Cada parte cumple una función específica para guiar el comportamiento del modelo:

- **Rol:** Define “quién” es el modelo en la interacción. Establecer un rol específico para el modelo orienta la voz, el nivel de experiencia y el tipo de razonamiento que debe adoptar durante la tarea.

Ejemplo: “Eres un experto en marketing digital.”

- **Contexto:** Sitúa la tarea en un marco de referencia. Brinda información relevante sobre el entorno, la situación o el objetivo global.

Ejemplo: “Trabajas para un startup de SaaS B2B.”

- **Tarea:** Especifica de manera clara y detallada qué debe hacer el modelo. Cuanto más específica y concreta sea la tarea, más relevante y enfocada será la respuesta.

Ejemplo: “Analiza este email y sugiere mejoras de redacción.”

- **Formato de Salida:** Establece la forma en que se espera recibir la respuesta, ya sea en una lista, tabla, JSON, texto, etc. Es esencial cuando la salida debe ser procesada automáticamente.

Ejemplo: "Devuelve una lista con 5 puntos específicos."

- **Restricciones:** Introduce limitaciones (como longitud máxima, tono de voz, idioma, etc.) que acotan el output para adaptarlo a requerimientos prácticos.

Ejemplo: “Máximo 200 palabras, tono profesional.”

- **Ejemplos:** Ofrecer ejemplos de input y output ayuda a clarificar expectativas, reduciendo ambigüedad y aumentando la consistencia del resultado.

Ejemplo: “Similar a este output:

- Mejora la claridad del mensaje.
- Asegúrate de que el asunto sea conciso”.

Al combinar los componentes anteriores, **un prompt completo podría verse de la siguiente manera**:

"Eres un experto en marketing digital. Trabajas para un startup de SaaS B2B. Analiza este email y sugiere mejoras. Devuelve una lista con 5 puntos específicos. Usa máximo 200 palabras y un tono profesional."

Por otro lado, **el modelo de output esperado sería**:

- Mejora la claridad del mensaje.
- Asegúrate de que el asunto sea conciso.

Luego de conocer esta estructura, **se puede comparar cómo cambia la calidad de la salida dependiendo de la estructura del prompt**:

Prompt	Output probable
"Haz sugerencias para este email".	Respuestas generales, a menudo incompletas, sin formato específico.
"Eres un experto en marketing digital. Trabajas para un startup de SaaS B2B. Analiza este email y sugiere mejoras. Devuelve una lista con 5 puntos específicos. Usa máximo 200 palabras y un tono profesional."	Una lista clara, con puntos específicos y en el tono adecuado, lista para ser utilizada directamente en un proceso automatizado.

Con esta base, ya puedes identificar y redactar prompts estructurados, elemento esencial antes de pasar a técnicas avanzadas de prompting.

🤔 **Para Pensar**

¿Qué tipo de respuesta crees que daría el modelo si le pides "Mejora este email" versus si usas la estructura completa?

¿Dónde sería más fácil integrar esa salida en un proceso automatizado?

Micro-práctica: Identifica y mejora prompts

Lee los siguientes ejemplos y reflexiona:

❌ **Prompt débil**

"Revisa este texto."

Problemas:

- No define tarea, formato, ni contexto.
- El output será impredecible.

✅ **Prompt Mejorado**

Eres un editor profesional. Analiza el siguiente email para detectar áreas de mejora. Devuelve una lista con 3 sugerencias concretas. Usa un tono constructivo.

Ventajas:

- Rol claro, tarea específica, formato definido.

Solución explicada:

Al incorporar cada uno de los componentes descritos, la IA comprende exactamente qué debe hacer, cómo debe hacerlo y en qué formato debe presentar la información. Esta claridad es esencial para obtener resultados consistentes, confiables y útiles, lo que a su vez permite integrar esos outputs en sistemas automatizados sin complicaciones.

Técnicas Avanzadas de Prompt Engineering

Técnicas Avanzadas de Prompting: del concepto a la práctica

En el siguiente video vas a aplicar en vivo las diferentes técnicas de prompting y entenderás qué transforma cada una en la salida del modelo.

Una vez que has dominado la estructura básica de un prompt efectivo, el siguiente paso esencial es incorporar **técnicas avanzadas que te permitan controlar el razonamiento del modelo, así como la calidad y el formato de las respuestas generadas**.

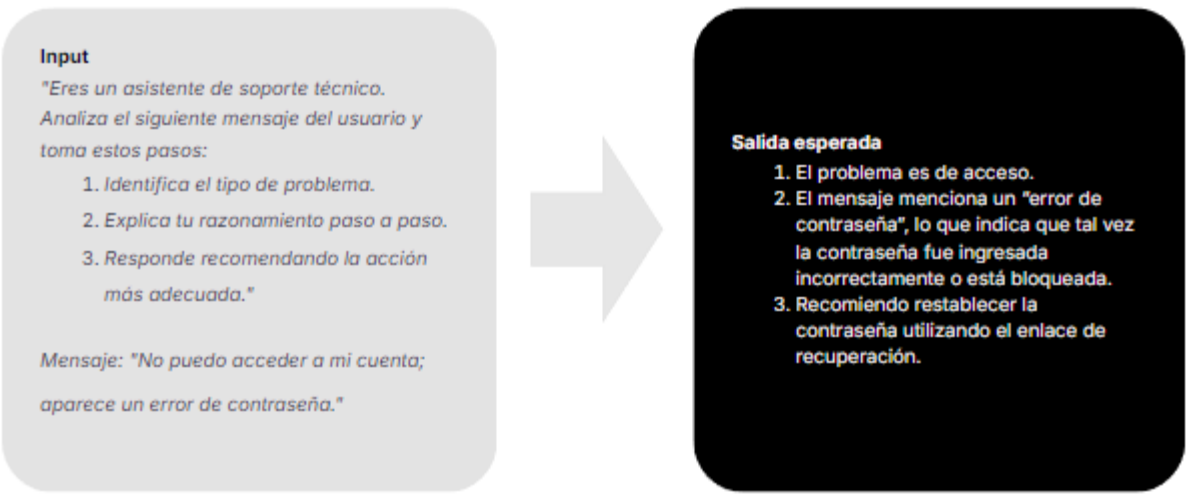
Estas técnicas son especialmente valiosas en contextos donde la **precisión, la consistencia y la automatización son cruciales**, como en flujos de trabajo integrados o tareas de procesamiento de información compleja.

A continuación, explicaremos las principales técnicas avanzadas utilizadas en el prompt engineering profesional para que comprendas en qué casos conviene usar cada una de ellas.

👉 Chain of Thought (Cadena de Pensamiento)

La técnica de **Chain of Thought (CoT)** consiste en instruir explícitamente al modelo para que razone sus respuestas paso a paso, en lugar de dar directamente una respuesta final. Esto ayuda a mejorar la transparencia y la precisión, especialmente en tareas que requieren lógica o descomponer problemas complejos.

Ejemplo de Prompt CoT



¿Cuándo usarlo?

Esta técnica es útil para:

- Procesos de toma de decisiones.

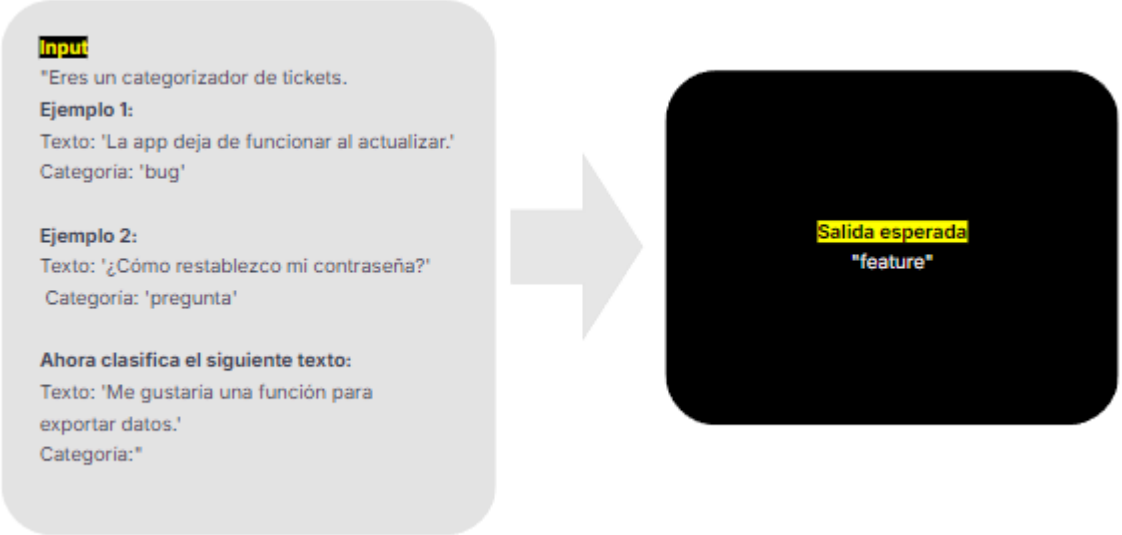
- Problemas de lógica o matemáticos.
- Extracción de información a partir de textos extensos o ambiguos.

Se destaca por generar **respuestas más claras y útiles para la revisión humana, facilita la depuración y mejora la reproducibilidad**. Sin embargo, si no se filtra adecuadamente, puede generar respuestas largas e imprácticas que afectan la eficiencia del proceso.

👉 Few-shot Learning

El Few-shot Learning es una técnica en la que **se proporcionan al modelo varios ejemplos concretos (inputs y outputs esperados) junto con la tarea a realizar**. Esto ayuda a reducir la ambigüedad y guía al modelo hacia el comportamiento deseado, mejorando la precisión en tareas poco frecuentes o complejas.

A continuación, un Ejemplo de Prompt Few-shot learning:



¿Cuándo usarlo?

- Al adaptar el modelo a tareas específicas sin necesidad de ajuste fino.
- Cuando la tarea tiene excepciones o patrones sutiles que pueden ilustrarse mediante ejemplos.

Esta capacidad permite una rápida adaptación a nuevas tareas con pocos ejemplos y reduce errores en situaciones con múltiples interpretaciones.

Sin embargo, si los ejemplos son sesgados o no representativos, puede generar errores. Además, los prompts largos consumen más tokens, lo que puede ser problemático si hay un límite.

👉 Role Playing

El Role Playing asigna explícitamente al modelo un rol específico (ej. "Eres un abogado especializado en contratos"). Este enfoque ajusta el tipo de lenguaje, el nivel técnico y el contexto, ayudando a alinear el output con las expectativas del usuario final.

Ejemplo de Prompt Role Playing:

"Eres un asesor de recursos humanos. Explica las consecuencias de llegar tarde repetidamente según el reglamento interno."

Salida esperada:

Una explicación formal sobre sanciones conforme políticas internas de RRHH.

¿Cuándo usarlo?

- Cuando es necesario generar salidas ajustadas a un dominio específico (legal, médico, educativo, etc.).
- Tareas donde el tono, formalidad o postura son críticos.

Permite obtener **resultados consistentes** tanto en formato como en terminología, adecuados para contextos específicos, y se puede combinar fácilmente con otras técnicas como Chain of Thought o Few-shot Learning. Sin embargo, **si el rol asignado es ambiguo o poco claro, el modelo puede desviarse**. Además, no reemplaza la necesidad de instrucciones técnicas o detalles sobre el formato de salida.

👉 Prompt Chaining

El Prompt Chaining es una técnica en la que **se dividen tareas complejas en pasos o prompts encadenados**, donde la salida de un modelo se utiliza como entrada para el siguiente. Permite descomponer flujos sofisticados en etapas manejables.

Ejemplo de Prompt Chaining:

1. **Prompt 1:** “Extrae los hechos clave de esta noticia en JSON.”
2. **Prompt 2:** “Usa el JSON anterior y genera un resumen de una frase para un boletín interno.”

Este esquema permite flexibilidad, ya que cada etapa se puede ajustar, evaluar o automatizar por separado.

¿Cuándo usarlo?

- Procesos multifase (ej. analizar, extraer y luego resumir información).
- Automatización de pipelines que requieren varias transformaciones consecutivas.

Facilita la trazabilidad y el control sobre cada paso, permitiendo integrar validaciones o reglas entre las etapas. Sin embargo, **a medida que se agregan más etapas, la complejidad de los flujos aumenta**, lo que puede generar una disminución en la velocidad de procesamiento, especialmente cuando el número de etapas es elevado.

Diseñando un flujo con Prompt Chaining

A continuación, vas a comprender cómo dividir una tarea compleja en pasos encadenados y a observar cómo cada prompt alimenta al siguiente hasta construir un flujo inteligente listo para automatizar.

Output Structuring y uso de JSON en Automatización

Una vez que tenemos una base sólida sobre la estructura de prompts y los fundamentos de las técnicas avanzadas de prompting, el siguiente paso es profundizar en **cómo solicitar y validar salidas estructuradas**. Esta habilidad es crucial cuando queremos que el output de un modelo de IA sea compatible con herramientas de automatización, APIs, sistemas de integración o pipelines empresariales.

El formato más comúnmente utilizado en este contexto es **JSON (JavaScript Object Notation)**, y en esta sección vamos a profundizar en cómo usarlo de manera efectiva para garantizar la compatibilidad con sistemas de automatización y APIs.

Pero ¿Qué es un **JSON**?, veamos.

JSON

Es un formato de intercambio de datos que se utiliza para almacenar y transportar información en una estructura de texto. Consta de pares clave-valor (como un diccionario o mapa) que representan datos.

Un ejemplo simple de un **objeto JSON** sería:

Copy

```
{ "nombre": "Juan",   "edad": 30,   "casado": true,
  "hijos": [
    {  "nombre": "Ana",  "edad": 5 },
    {  "nombre": "Carlos",  "edad": 3 }
  ]
}
```

En este ejemplo, "nombre", "edad", "casado" y "hijos" **son claves**.

Los valores asociados a cada clave pueden ser **cadenas de texto** (como "Juan"), **números** (como 30), booleanos (como true), o **listas** (como el array de "hijos").

El **JSON es fácil de leer y entender**, tanto para humanos como para sistemas, por lo que se utiliza ampliamente para la **comunicación entre aplicaciones o la integración de sistemas debido a su simplicidad y flexibilidad** para representar datos de manera estructurada.

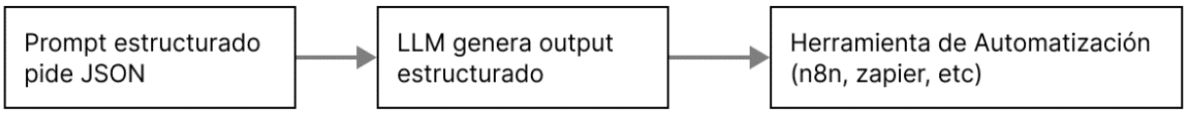
La importancia del output estructurado en flujos automatizados

En escenarios profesionales, rara vez basta con obtener una respuesta en lenguaje natural. Necesitamos garantizar que la IA responda de forma predecible y estructurada para que otras herramientas automáticas puedan interpretar y procesar fácilmente la información.

Por ejemplo, plataformas como n8n, Zapier o sistemas internos suelen “entender” datos en formato JSON, ya que:

- Permite **automatizar flujos**, extrayendo y reutilizando datos específicos.
- **Reduce el riesgo de errores** por salidas ambiguas o inconsistentes.
- **La información puede ser extraída y utilizada** para activar acciones en otros sistemas (como enviar un correo, actualizar un registro en una base de datos, etc.).
- Cuando el output es estructurado en JSON, resulta más sencillo hacer un **seguimiento y validar los resultados**.

Veamos el flujo para obtener un output estructurado:



Solicitar salidas en JSON es también una buena práctica para **reproducibilidad y trazabilidad, además de facilitar la validación automática de los resultados**.

Output Structuring

El **Output Structuring implica especificar el formato exacto de salida requerido**, como JSON, Markdown o CSV. Es fundamental cuando se busca integrar la respuesta del modelo en flujos automáticos o pipelines que dependen de datos estructurados, qué pueden ser procesados por máquinas.

Ejemplo de Prompt Output Structuring (JSON):

Responde solo en JSON. Dado el siguiente mensaje de usuario, clasifica según {'tipo':['bug','feature','pregunta']} y justifica. Mensaje: "No puedo adjuntar archivos a mi cuenta." Schema: {'tipo':string, 'justificacion':string}

Salida esperada

{"tipo": "bug", "justificacion": "el usuario no puede adjuntar archivos, lo cual es un fallo funcional"}

¿Cuándo usarlo?

- Integración con APIs, webhooks u otras herramientas automáticas.
- Cuando la respuesta efectiva debe seguir un schema estricto.

Genera un output **predecible, fácil de integrar y procesar**, minimizando errores humanos en etapas posteriores. Sin embargo, el prompt debe ser explícito; el modelo puede desviarse si no se especifica el formato exacto. Además, siempre es necesario validar el output antes de usarlo en producción.

¿Cómo estructurar un prompt para obtener salida en JSON?

Para lograr este objetivo, la redacción precisa del prompt es crítica. Es necesario especificar claramente el formato exacto de salida, incluyendo las claves que deben estar presentes, los tipos de datos que deben devolver y las instrucciones sobre cómo manejar casos en los que la información no sea aplicable o esté ausente.

Los elementos clave que debes incluir son:

- El **formato exacto solicitado** (JSON, incluyendo el schema y tipos de datos).
- La indicación explícita de que **se responda solamente en JSON**. Sin comentarios adicionales o explicaciones.
- **Restricciones o instrucciones** sobre el manejo de valores desconocidos o no aplicables (por ejemplo, usar null).

- **Parámetros del modelo** (como temperature y max_tokens) para controlar la creatividad, longitud y consistencia de la respuesta.

Plantilla recomendada de prompt para output JSON

Puedes utilizar micro-plantillas propias para asegurar outputs consistentes. Por ejemplo:

System: "Eres un asistente que responde exclusivamente en JSON válido."

User: "Clasifica el siguiente texto en {'categoria':['bug','pregunta','feature']}. Devuelve {'categoria':string,'urgencia':string,'razon':string}. Si no aplica, usa null en los campos correspondientes. Responde sólo con JSON."

Esta plantilla define de forma clara tanto la estructura deseada del output como las expectativas ante casos límite.

Ejemplo práctico

Para hacer más claro cómo funciona esto en la práctica, supongamos que damos el siguiente input al modelo: **"Mi app se cierra al abrir la configuración.".**

Un posible prompt ejemplo: *Clasifica el siguiente texto en {'categoria':['bug','pregunta','feature']}. Devuelve {'categoria':string,'urgencia':string,'razon':string}. Si no aplica, usa null. Texto: "Mi app se cierra al abrir la configuración". Responde sólo con JSON.*

Output esperado

{"categoria": "bug", "urgencia": "alta", "razon": "crash al abrir configuración"}

🤔 **Para pensar**

¿Qué podría ocurrir si omites la instrucción "Responde solo con JSON"? ¿Cómo impactaría esto la automatización?

Si no incluyes la instrucción "Responde solo con JSON", el modelo podría devolver una respuesta en lenguaje natural. Esto no sería útil en un flujo automatizado porque no es procesable por otros sistemas.

¿Por qué importa la validación de salidas?

Validar la salida significa ejecutar el prompt varias veces (al menos 3), comprobando que el resultado sea:

- Parseable como JSON (sin errores de sintaxis).
- Que contenga exactamente las claves requeridas.

- Que respete el formato incluso ante inputs límite.

Una buena práctica **es incluir una cláusula de fallback (uso de null)** previene salidas vacías o inconsistentes ante casos no cubiertos en los ejemplos.

Modelos que permiten definir la estructura del Output como un Input Adicional

Con la evolución de modelos como **Google Gemini y OpenAI GPT-4**, ahora es posible definir la estructura del resultado mediante un input adicional, en lugar de incluir esa instrucción dentro del prompt principal.

Esta mejora permite separar la forma de la respuesta del contenido de la consulta, aportando mayor **modularidad, flexibilidad y facilidad de integración en flujos automatizados**.

Tradicionalmente, los usuarios debían especificar dentro del prompt cómo querían que se formateara la salida, por ejemplo, pidiendo: “responde en formato JSON con los campos título, resumen y categoría”. En cambio, **los modelos más recientes permiten indicar el formato deseado mediante un parámetro adicional, como output_format, sin sobrecargar el prompt con instrucciones de formato.**

Por ejemplo:

- Prompt principal: “Analiza este texto y extrae los temas más relevantes.”
- Input adicional (output_format): {"formato": "JSON", "estructura": ["tema", "relevancia"]}

El modelo generará directamente una respuesta con esa estructura, lista para ser procesada por otro sistema.

Para experimentar esta funcionalidad se requiere plataformas que soporten modelos avanzados y parámetros personalizados, como, por ejemplo:

- **Google AI Studio**: ofrece una interfaz intuitiva para definir output_format en Gemini, facilitando su integración en pipelines automatizados.
- **n8n**: permite conectar modelos de OpenAI o Gemini dentro de flujos de trabajo, especificando el formato de salida directamente en los nodos (ideal para tareas como clasificación de tickets o análisis de texto).

Caso Práctico

Tradicionalmente, el formato de salida se especificaba directamente en el prompt.

- **Prompt**

Eres un asistente experto en soporte técnico que responde únicamente en JSON válido.

Clasifica el siguiente texto según {'categoria': ['bug', 'pregunta', 'feature']}.

Devuelve únicamente: {'categoria': string, 'urgencia': string, 'razon': string}.

Si no aplican las categorías, usa null en los respectivos campos.

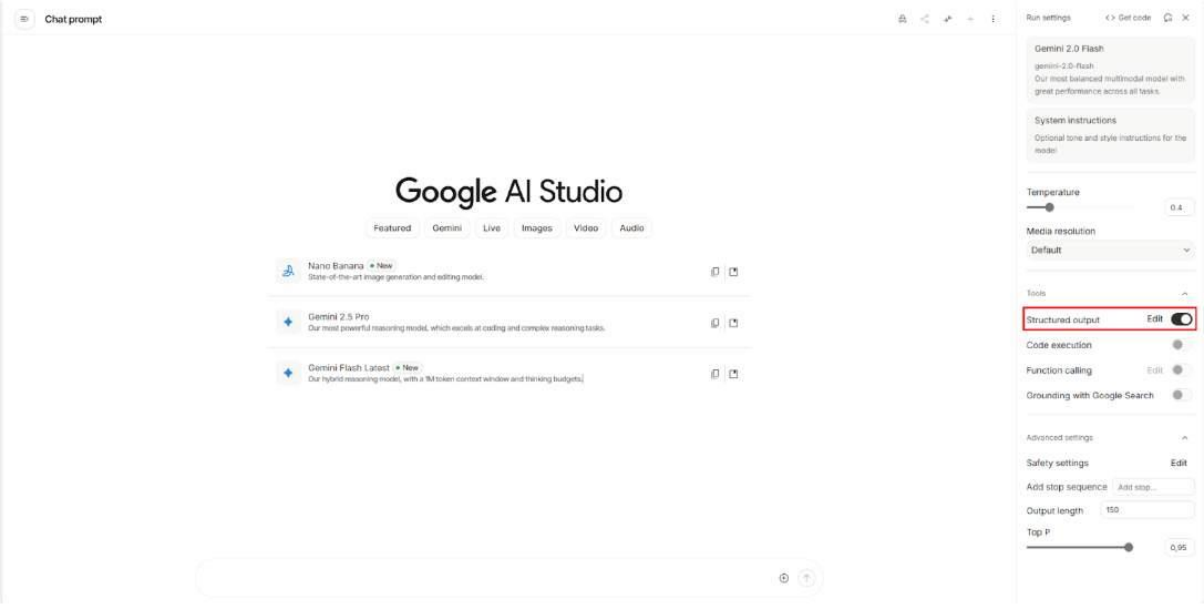
- **Output**

{"categoria":"bug", "urgencia":"alta", "razon":"crash al iniciar sesión"}

Texto: "Mi app se cierra al abrir el panel de configuración"

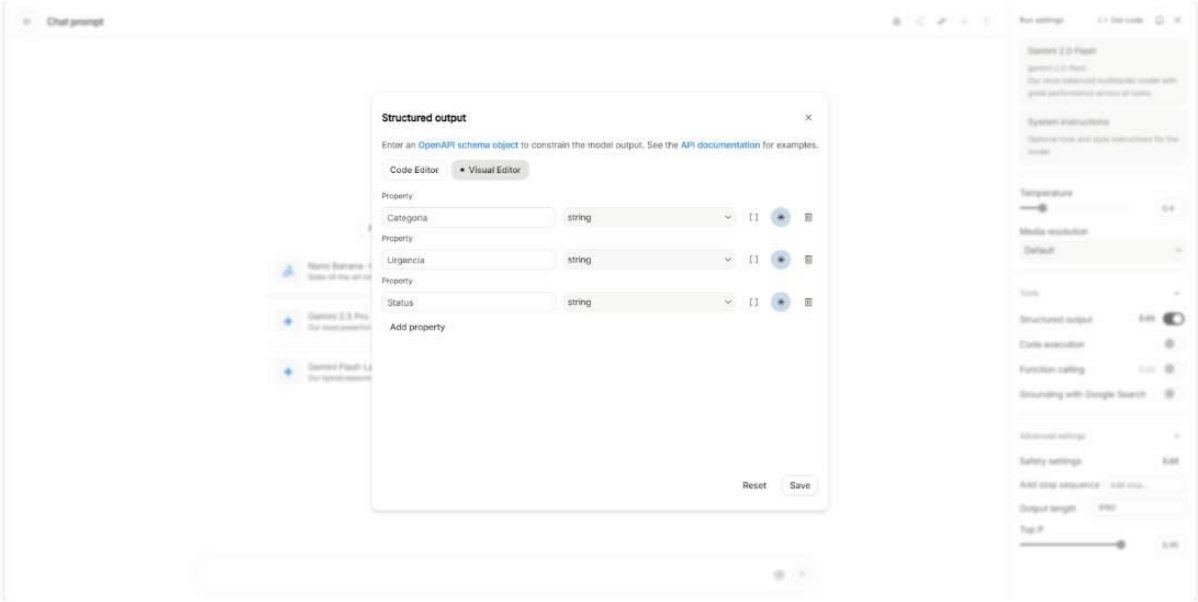
En este caso, el contexto, la tarea y la estructura esperada se incluye dentro del mismo prompt.

Por otro lado, en Google AI Studio definis la estructura del output desde el entorno, usando parámetros como output_format. Así, el prompt puede centrarse en el contenido y la tarea, mientras que la estructura se configura por separado.



En la pantalla principal de **Google AI Studio** puedes activar la opción para definir un **Structured Output**.

Luego, especificas la estructura deseada (por ejemplo, los campos **categoria, urgencia y razón**).



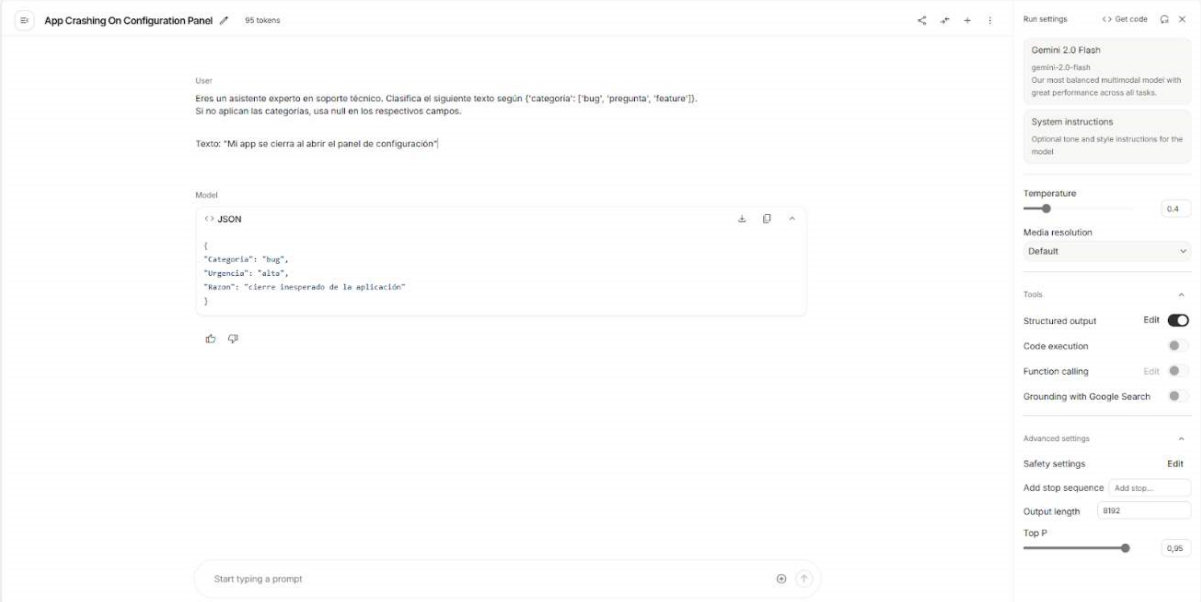
Finalmente, el **prompt se simplifica** a algo como:

Eres un asistente experto en soporte técnico. Clasifica el siguiente texto según {'categoria': ['bug', 'pregunta', 'feature']}.

Si no aplican las categorías, usa null en los respectivos campos.

Texto: "Mi app se cierra al abrir el panel de configuración"

El resultado esperado:



Casos Prácticos y Evaluación de Prompts en Producción

Ahora exploraremos cómo las técnicas avanzadas de prompting se aplican y evalúan en escenarios industriales reales. **Veremos tres casos de uso** representativos en **soporte técnico, reportes ejecutivos y salud**, donde se implementan estrategias de diseño de prompts estructurados y técnicas avanzadas como Chain of Thought y Few-shot learning.

Además, se detallará cómo medir el éxito de estas soluciones mediante KPIs (Indicadores Clave de Desempeño), que son métricas específicas utilizadas para evaluar el rendimiento de un proceso o solución en relación con los objetivos establecidos. Estos KPIs permitirán medir de manera puntual el impacto tangible que tiene un buen diseño de prompt frente a enfoques más simples, proporcionando datos concretos sobre la efectividad y optimización del proceso, así como guiar su mejora continua.

👉 Caso 1: Soporte Técnico — Clasificación Automática de Tickets

Problema: el equipo de soporte de una empresa recibe cientos de tickets diarios. Clasificarlos manualmente por categoría y urgencia es costoso y genera retrasos significativos en la atención al cliente.

Solución con Prompt Avanzado Estructurado: se utiliza un prompt que define explícitamente el rol de la IA, el contexto del ticket, las categorías posibles, y establece un formato estricto en JSON, incluyendo instrucciones claras para el manejo de casos no clasificables (cuando un ticket no encaja en ninguna categoría).

Prompt

System: "Eres un agente experto en soporte tecnico."

User: "Clasifica el siguiente ticket en {'categoria': ['bug', 'consulta', 'feature request']} segun el texto. Devuelve un JSON con las claves {'categoria': string, 'urgencia': string, 'razon': string}. Si el ticket no coincide con ninguna categoria, usa 'null' en todas las claves. Ticket: 'No puedo ingresar con mi usuario, me da error de autenticacion.'"

Output esperado

Copy

```
{
  "categoria": "bug",
  "urgencia": "alta",
  "razon": "error de autenticación impide acceso"
```

}

KPIs y evaluación:

- *KPI*: Porcentaje de tickets correctamente clasificados y parseables como JSON válido.
- *Resultado Medible*: En 10 pruebas, el 90% de los tickets generan JSON válido con clasificaciones útiles.

Impacto: El ruteo inicial de tickets se automatiza en un 85%, lo que reduce el tiempo de respuesta promedio en un 40%.

La estructura estricta en JSON permite una clasificación automática precisa y eficiente de tickets, lo que ahorra tiempo y mejora la calidad de la atención.

👉 Caso 2: Reportes Ejecutivos — Generación Estructurada de Insights

Problema: los directivos necesitan resúmenes semanales sobre tendencias en ventas. Sin embargo, las respuestas generadas manualmente o mediante prompts libres suelen tener un formato inconsistente y omiten datos clave, lo que dificulta la toma de decisiones.

Solución con prompt con Output Structuring y Chain of Thought: prompt que especifica el formato Markdown, estructura de puntos y un razonamiento paso a paso para claridad:

Prompt

System: "Eres un analista de datos senior."

User: "Resume las ventas de la semana siguiendo estos pasos: primero identifica las principales tendencias, luego enumera 3 riesgos emergentes. Devuélvelo en Markdown con dos secciones claras ('Tendencias principales' y 'Riesgos emergentes') y usa viñetas en cada una."

Output esperado

Copy

Tendencias principales

- Aumento del 15% en ventas online respecto a la semana anterior.

- Incremento en compras desde dispositivos móviles.

- Disminución en la conversión de nuevos usuarios.

Riesgos emergentes

- Saturación de inventario en almacén central.

- Descenso leve de satisfacción posventa.

- Incremento de devoluciones en línea.

KPIs y evaluación:

- *KPI*: Porcentaje de reportes que cumplen completamente con el formato y presentan todos los datos clave.
- *Resultado Medible*: 8 de 10 reportes cumplen con los criterios formales y de contenido definidos.

Impacto: Los ejecutivos pueden revisar y comparar reportes un 30% más rápido, reduciendo las revisiones manuales.

El uso de Chain of Thought y una salida estructurada en Markdown permite proporcionar insights claros y fáciles de consumir para los ejecutivos.

👉Caso 3: Salud — Extracción de Información Clave en Reportes Clínicos

Problema: los hospitales deben extraer diagnósticos, tratamientos y advertencias de reportes médicos libres para alimentar sistemas de seguimiento. Los prompts no estructurados a menudo pierden datos relevantes o no son consistentes, lo que dificulta la integración automática en sistemas de alerta clínica.

Solución con técnica Few-shot + formato estructurado (JSON): prompt con varios ejemplos para afinar el reconocimiento y formato de salida, más un esquema estricto de campos:

Prompt

System: "Eres un asistente clínico digital."

User: "Extrae {'diagnostico': string, 'tratamiento': string, 'advertencia': string o null} del siguiente reporte médico:

Input: 'Paciente refiere dolor toracico, realiza ECG, diagnostico: angina estable, inicia aspirina 100mg/dia.'

Output: {'diagnostico': 'angina estable', 'tratamiento': 'aspirina 100mg/dia', 'advertencia': null}

Ejemplo:

Input: 'Paciente con infección respiratoria aguda. Se prescribe amoxicilina. Alergico a penicilinas.' Output: {'diagnostico': 'infección respiratoria aguda', 'tratamiento': 'amoxicilina', 'advertencia': 'alergia a penicilinas'} Ahora analiza el reporte: 'Paciente consulta por fiebre y tos.

Diagnostico: bronquitis aguda.

Tratamiento: ibuprofeno 400mg cada 8 hs."

Output esperado

Copy

{ "diagnostico": "bronquitis aguda", "tratamiento": "ibuprofeno 400mg cada 8 hs", "advertencia": null }

KPI y evaluación:

- *KPI*: Precisión y completitud en la extracción de campos clave; % de outputs JSON válidos y parseables.
- *Resultado medible*: Con técnica Few-shot, la precisión sube de 70% (prompt base) a 88% (mejora ≥15%).

Impacto: Reducción de errores de transcripción y aumento de registros digitales confiables para sistemas de alerta clínica.

La técnica **Few-shot permite mejorar la extracción de información clave** de los reportes médicos, aumentando la precisión y reduciendo los errores en los sistemas de seguimiento clínico.

Checklist y criterios operativos

No solo se trata de escribir prompts, sino también de evaluarlos y mejorarlos mediante evidencia real y continua. Este proceso asegura que tengas en cuenta todos los aspectos clave para diseñar un prompt profesional y efectivo.

Al finalizar cada prompt, responde:

- ¿Defines en el prompt el rol del modelo (ej: experto, asistente)?
- ¿Indicas explícitamente el formato de salida (ej: solo JSON)?
- ¿Incluyes ejemplos o instrucciones "qué hacer si no aplica" (fallback/null)?
- ¿El output es parseable y estructurado en 2 de 3 ejecuciones?
- ¿Justificas tus parámetros (temperature, max_tokens)?

Cierre

¡Vamos a recapitular todo lo que hemos aprendido!

En esta clase revisamos **cómo la IA potencia la automatización inteligente y permite que los sistemas interpreten, decidan y se adapten**.

Abordamos los **fundamentos de los LLMs y cómo parámetros como temperature y max_tokens afectan creatividad, precisión y extensión de las respuestas**.

Finalmente, profundizamos en el **Prompt Engineering**, sus componentes esencialesy técnicas avanzadas de prompting para mejorar la calidad y consistencia de los resultados.

Principales aprendizajes para llevarte:

- La IA amplía la automatización mediante adaptación y toma de decisiones.
- Los LLMs permiten personalizar resultados ajustando sus parámetros.
- Un prompt bien diseñado guía al modelo hacia respuestas precisas y útiles.
- Las técnicas avanzadas de prompting mejoran desempeño en tareas complejas.
- La experimentación iterativa es esencial para refinar prompts y parámetros.
- Integrar salidas estructuradas (JSON, Markdown, etc.) facilita la automatización.
- Revisa tus prompts y valida que cumplan las buenas prácticas vistas en clase.
- Experimenta en un playground variando temperature y max_tokens para observar su impacto.

🤔 **Para pensar**

¿Cómo cambiarías tu prompt si la IA comenzara a fallar en devolver el formato correcto o a confundir el contexto de la tarea?

A través de práctica y análisis crítico, desarrollarás la capacidad de diseñar prompts robustos y optimizar resultados de IA en entornos reales.

Homework

Homework 1: Fundamentos de IA para Automatización

En esta homework podrás experimentar con distintos parámetros poniendo en juego sus funciones y comparando resultados.

Ejercicio A — Experimento de parámetros: temperature vs. coherencia

Usando el mismo prompt realiza 2 ejecuciones variando únicamente temperature (0.0 y 0.9). Documenta prompt, parámetros usados (temperature, max_tokens, model), la salida textual o captura y calcula el coste estimado por ejecución. Presenta ambas ejecuciones en la tabla y comenta brevemente las diferencias observadas.

Prompt sugerido (puede usarse tal cual):

"Resume en una frase: El producto presentó fallos intermitentes tras la actualización y varios usuarios reportaron pérdida de datos menores. Equipo técnico trabajando en parche."

Configuración de ejemplo:

- Ejecución A: model = gpt-4 (o similar), temperature = 0.0, max_tokens = 60
- Ejecución B: model = gpt-4, temperature = 0.9, max_tokens = 60

Completa la siguiente tabla

Parametro	Ejecución A	Ejecución B
Modelo		
Temperature		
max_tokens		
Prompt		
Salida		

Parametro	Ejecución A	Ejecución B
Tokens usados		
Costo estimado		

Ejercicio B — Comparativa y recomendación

Elija un caso simple: "Clasificación de short_feedback (≤50 palabras) en {positivo, neutro, negativo}" y diseñe una recomendación sobre qué modelo y configuración usar para una integración en un workflow automatizado (p. ej. n8n). Prepare:

- Una tabla con 3 salidas de ejemplo (usar 2 modelos distintos y 2 settings de temperature). Cada fila debe contener: modelo, temperature, prompt breve, salida (texto o captura), costo estimado por ejecución.
- Un análisis breve (máx 1 página) que justifique la elección del modelo y la configuración para la tarea de clasificación. Incluya al menos un criterio cuantificable (p. ej. coste estimado por 1k tokens o limitación de context window) y explique cómo ese criterio afecta la decisión operativa.

Completa la siguiente tabla

Parametros	Modelo 1	Modelo 2	Modelo 3
Modelo			
Temperature			
Prompt			
Salida			
Tokens usados			
Coste estimado			
Observaciones			

Homework 2: Calculando el Costo de Uso de IA

En esta oportunidad el objetivo es tokens y costo por uso para estimar gastos al integrar IA en procesos automatizados.

Consigna

1. Considerá que una empresa de soporte técnico genera 100 respuestas automáticas por día con un promedio de 300 tokens por respuesta (150 tokens de entrada + 150 tokens de salida).

2. Utiliza la siguiente tabla de precios y calcula el costo diario y mensual estimado para cada modelo. Aplica el análisis para los modelos de GPT-4.1, o4–mini y GPT-5.
3. Analizá cuál resulta más conveniente en función del volumen y la calidad esperada.

Modelo	Costo de entrada	Entrada en caché	Costo de Salida
GPT-4.1	USD 3,00/1 millón de tokens	USD 0,75/1 millón de tokens	USD 12,00/1 millón d
o4–mini	USD 4,00/1 millón de tokens	USD 1,00/1 millón de tokens	USD 16,00/1 millón d
GPT-5	USD 1,250/1 millón de tokens	USD 0,125/1 millón de tokens	USD 10,000/1 millón

Los costes se han obtenido del sitio oficial de OpenAI (https://openai.com/es-419/api/pricing/(opens in a new tab))

Homework 3: Dominando el Prompt Engineering

En esta homework vas a aplicar técnicas avanzadas de prompt engineering para transformar prompts básicos en herramientas precisas, profesionales y orientadas a resultados.

Consigna

Vas a trabajar sobre un caso laboral simulado en el área de marketing digital. Tu tarea será **diseñar, optimizar y comparar** distintos prompts aplicando las técnicas vistas en clase.

Paso 1 - Diagnóstico inicial

1. Leé el **Caso base** incluido más abajo.
2. Identificá cuál sería el **objetivo general** del prompt (qué se quiere lograr).
3. Redactá un **prompt inicial genérico** (sin estructura ni técnicas) que apunte a ese objetivo.
4. Ejecutá ese prompt en ChatGPT u otro modelo (solo a modo de práctica) y analizá los problemas que presenta la respuesta (ambigüedad, falta de foco, formato, tono, etc.).

Paso 2 - Diseño profesional del prompt

1. Redactá un **prompt estructurado** que incluya:
    - Rol**:** ¿Qué papel debe desempeñar la IA?

- Contexto**:** ¿Qué contexto debe tener en cuenta la IA?
- Tarea**:** ¿Qué acción debe realizar la IA?
- Formato**:** ¿Cómo debe presentarse la respuesta?
- Restricciones**:** ¿Qué límites deben considerarse en la respuesta (tono, longitud, idioma, etc.)?
- Ejemplos (si aplica): ¿qué ejemplos pueden ayudar a guiar a la IA?

2. Explicá en pocas líneas por qué cada parte de la estructura fue definida de esa manera.

Paso 3 - Aplicación de Técnicas Avanzadas

Ahora, vas a aplicar diferentes técnicas avanzadas de prompt engineering a los escenarios planteados. Para cada uno, deberás desarrollar un prompt optimizado utilizando distintas técnicas. Al final, compara los resultados obtenidos.

Caso A: "Mejorar el impacto del asunto del email"

Objetivo: Mejorar el asunto del correo para captar la atención del CTO.

Desarrolla un prompt que genera 5 opciones de asunto para un correo dirigido a un CTO de una empresa mediana que busca mejorar la eficiencia del equipo de desarrollo con la herramienta SaaS DataSync. Justifica brevemente por qué cada asunto es adecuado, enfocándote en los valores clave para un CTO: productividad, ahorro de tiempo, y eficiencia.

Caso B: "Adaptación del email según etapa del embudo"

Objetivo: Generar diferentes versiones de un correo adaptadas a las diferentes etapas del embudo de ventas: awareness, consideration, decision.

Desarrolla un prompt que incluya ejemplos de correos anteriores para cada etapa del embudo. Luego, pide a la IA que genere nuevos correos basados en esos ejemplos.

Caso C: "Personalización según el público objetivo"

Objetivo: Personalizar un correo para tres roles diferentes: CTO, Data Engineer e IT Manager.

Desarrolla un prompt que crea tres versiones de un correo dirigido a los siguientes roles en una empresa: CTO, Data Engineer e IT Manager. Cada correo debe resaltar el valor de DataSync según las prioridades de cada rol (eficiencia, manejo de grandes datos, seguridad, escalabilidad). Usa un tono profesional y directo.

Caso D: "Optimización del formato y estructura del correo"

Objetivo: Generar un correo optimizado en términos de formato, estructura y claridad.

Elabora un prompt que genera un correo estructurado para un CTO con los siguientes elementos:

1. **Asunto**: Claro y conciso.
2. **Introducción**: Breve, enfocada en el problema del cliente.
3. **Cuerpo**: Beneficios de DataSync, utilizando viñetas o subtítulos.
4. **Llamado a la acción**: Claro y directo.

Asegúrate de que el tono sea profesional y enfocado en los beneficios del producto. El output debe ser un JSON para que se pueda integrar a una automatización de envío automático de correos.

Caso E: "Generación de un flujo de correos"

Objetivo: Crear una secuencia de correos automatizados que sigan un flujo lógico desde el primer contacto hasta la conversión.

Elabora un prompt que crea una secuencia de tres correos automatizados para las etapas del embudo de ventas (Awareness, Consideration, Decision) para **DataSync**. Cada correo debe ser breve, persuasivo y enfocado en mover al prospecto hacia la siguiente etapa. El primer correo debe introducir el producto, el segundo debe profundizar en sus beneficios, y el tercero debe incluir una oferta o llamada a la acción clara.

Una vez hayas completado cada mini caso y aplicado las técnicas de prompt engineering, realiza una comparación entre las respuestas obtenidas, a partir de las siguientes preguntas:

- ¿Qué cambios notaste en la calidad o formato de la respuesta?
- ¿Qué técnica produjo el resultado más útil y por qué?
- ¿En qué contexto es más efectivo aplicar cada técnica?

Material Complementario

Material complementario HW3

Material complementario 1

Caso base: "Campaña de Email Marketing B2B"

Eres parte del equipo de marketing de una startup llamada **DataSync**, que ofrece un software de integración de datos para empresas medianas (SaaS B2B).

La empresa quiere mejorar su **tasa de apertura y conversión** en las campañas de correo dirigidas a potenciales clientes corporativos.
Actualmente, los correos que se envían son demasiado genéricos y no logran captar interés.

Ejemplo del email actual:

Asunto: Mejora tu productividad con DataSync

Cuerpo:
Hola,
DataSync te permite integrar fácilmente todos tus datos en un solo lugar. Conecta tus herramientas y obtén informes más rápidos.

Agenda una demo gratuita hoy.

El director de marketing te pide que uses IA para **mejorar el enfoque y el tono de los emails**, adaptándolos a distintos públicos y etapas del embudo de ventas (awareness, consideración, decisión).

Algunos de los últimos emails que se han usado y probado en el embudo:

- **Awareness**: "¿Sabías que puedes integrar todos tus datos sin esfuerzo? DataSync te ofrece una forma fácil y rápida de hacerlo. Agenda una demo para conocer más."
- **Consideration**: "DataSync mejora la productividad de tu equipo de desarrollo al integrar datos de múltiples fuentes en tiempo real. Descubre cómo podemos ayudarte."
- **Decision**: "DataSync es la herramienta que tu equipo necesita para mejorar la eficiencia y productividad. ¡Solicita una demo gratuita ahora y empieza a ahorrar tiempo!"

Material complementario 2

Contexto operativo (para incluir en tus prompts)

- **Tipo de producto:** SaaS B2B (plataforma de integración de datos).
- **Público objetivo:** responsables de tecnología (CTO, Data Engineers, IT Managers).
- **Objetivo de comunicación:** aumentar las tasas de apertura y clics en los emails.

- **Restricciones:** tono profesional, mensajes breves y centrados en el valor para el cliente.
- **Idiomas permitidos:** español o inglés.